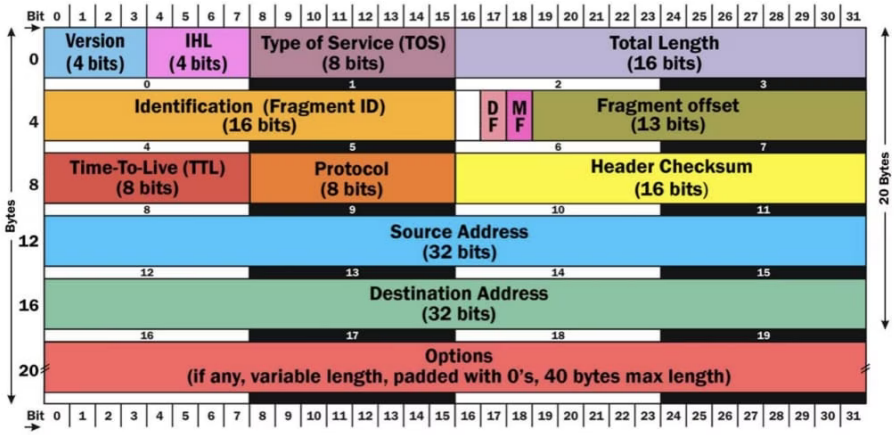
DANIEL MIESSLER

# A tcpdump Tutorial with Examples

*50 ways to isolate traffic for cybersecurity, network administration, and other technical roles*

January 5, 2004

#cybersecurity #technology #tutorial #top




`tcpdump` is the world's premier network analysis tool—combining both power and simplicity into a single command-line interface. This guide will show you how to use it.

tcpdump is a powerful command-line packet analyzer. It allows you to capture and inspect network traffic in real-time. This tool is invaluable for network administrators, security professionals, and anyone who needs to understand network behavior.

In this tutorial, we'll explore 50 practical examples of using `tcpdump`. These examples will cover a wide range of use cases, from basic traffic capture to advanced filtering and analysis.

## BASIC SYNTAX

The basic syntax of `tcpdump` is:

```bash
tcpdump [options] [expression]
```

- `options`: Modify the behavior of `tcpdump`, such as specifying the interface to capture on or the output format.
- `expression`: Defines what kind of traffic to capture. This is where you specify hostnames, IP addresses, ports, protocols, and other criteria.

## CAPTURING TRAFFIC ON AN INTERFACE

To capture all traffic on a specific interface, use the `-i` flag followed by the interface name. For example, to capture traffic on the `eth0`

DANIEL MIESSLER

```bash
tcpdump -i eth0
```

To see a list of all available interfaces, use the command:

```bash
tcpdump -D
```

### CAPTURING TRAFFIC TO/FROM A SPECIFIC HOST

To capture traffic to or from a specific host, use the `host` keyword followed by the hostname or IP address:

```bash
tcpdump host 192.168.1.100
```

This will capture all traffic to and from the host with the IP address `192.168.1.100`.

### CAPTURING TRAFFIC ON A SPECIFIC PORT

To capture traffic on a specific port, use the `port` keyword followed by the port number:

```bash
tcpdump port 80
```

This will capture all traffic on port 80 (HTTP).

### COMBINING FILTERS

You can combine filters using `and`, `or`, and `not` operators. For example, to capture all traffic to or from host `192.168.1.100` on port 80, use:

```bash
tcpdump host 192.168.1.100 and port 80
```

DANIEL MIESSLER

```bash
tcpdump src host 192.168.1.100 and \( port 80 or port 443 \)
```

## ADVANCED FILTERING

### FILTERING BY PROTOCOL

To filter by protocol, use the `ip`, `tcp`, `udp`, or other protocol keywords. For example, to capture only TCP traffic:

```bash
tcpdump tcp
```

To capture only UDP traffic:

```bash
tcpdump udp
```

### FILTERING BY SOURCE OR DESTINATION

To filter by source or destination host or port, use the `src` or `dst` keywords:

```bash
tcpdump src host 192.168.1.100
```

This will capture all traffic from the host `192.168.1.100`.

```bash
tcpdump dst port 443
```

This will capture all traffic destined for port 443.

### FILTERING BY NETWORK

To capture traffic within a specific network, use the `net` keyword:

DANIEL MIESSLER

This will capture all traffic within the 192.168.1.0/24 network.

### SAVING CAPTURED TRAFFIC TO A FILE

To save captured traffic to a file, use the `-w` flag followed by the filename:

```bash
tcpdump -w capture.pcap -i eth0
```

This will save all captured traffic on the `eth0` interface to the file `capture.pcap`.

You can later analyze this file using `tcpdump` or another packet analyzer like Wireshark.

### READING CAPTURED TRAFFIC FROM A FILE

To read captured traffic from a file, use the `-r` flag followed by the filename:

```bash
tcpdump -r capture.pcap
```

This will read and display the traffic from the file `capture.pcap`.

### VERBOSITY

You can control the verbosity of `tcpdump` output using the `-v`, `-vv`, or `-vvv` flags.

- `-v`: Verbose output.
- `-vv`: More verbose output.
- `-vvv`: Most verbose output.

For example:

DANIEL MIESSLER

## 50 TCPDUMP EXAMPLES

Here are 50 `tcpdump` examples to help you isolate traffic in various situations:

**Capture all traffic on interface `eth0`:**

```bash
tcpdump -i eth0
```

**Capture all traffic on interface `wlan0`:**

```bash
tcpdump -i wlan0
```

**Capture traffic to or from host `192.168.1.100`:**

```bash
tcpdump host 192.168.1.100
```

**Capture traffic to or from host `example.com`:**

```bash
tcpdump host example.com
```

**Capture traffic on port 80 (HTTP):**

```bash
tcpdump port 80
```

**Capture traffic on port 443 (HTTPS):**

```bash
tcpdump port 443
```

**Capture traffic on port 22 (SSH):**

DANIEL MIESSLER

### Capture traffic on port 21 (FTP):

```bash
tcpdump port 21
```

### Capture traffic on port 25 (SMTP):

```bash
tcpdump port 25
```

### Capture traffic on port 53 (DNS):

```bash
tcpdump port 53
```

### Capture traffic from host `192.168.1.100`:

```bash
tcpdump src host 192.168.1.100
```

### Capture traffic to host `192.168.1.100`:

```bash
tcpdump dst host 192.168.1.100
```

### Capture traffic from port 80:

```bash
tcpdump src port 80
```

### Capture traffic to port 443:

```bash
tcpdump dst port 443
```

DANIEL MIESSLER

```bash
tcpdump tcp
```

### Capture all UDP traffic:

```bash
tcpdump udp
```

### Capture all ICMP traffic:

```bash
tcpdump icmp
```

### Capture traffic to or from network `192.168.1.0/24`:

```bash
tcpdump net 192.168.1.0/24
```

### Capture traffic from network `192.168.1.0/24`:

```bash
tcpdump src net 192.168.1.0/24
```

### Capture traffic to network `192.168.1.0/24`:

```bash
tcpdump dst net 192.168.1.0/24
```

### Capture traffic to host `192.168.1.100` on port 80:

```bash
tcpdump dst host 192.168.1.100 and dst port 80
```

### Capture traffic from host `192.168.1.100` on port 443:

DANIEL MIESSLER

### Capture traffic to or from host `192.168.1.100` on port 80 or 443:

```bash
tcpdump host 192.168.1.100 and \( port 80 or port 443 \)
```

### Capture all traffic except ICMP:

```bash
tcpdump not icmp
```

### Capture all traffic except port 80:

```bash
tcpdump not port 80
```

### Capture traffic with a specific TCP flag (SYN):

```bash
tcpdump 'tcp[tcpflags] & tcp-syn != 0'
```

### Capture traffic with a specific TCP flag (ACK):

```bash
tcpdump 'tcp[tcpflags] & tcp-ack != 0'
```

### Capture traffic with a specific TCP flag (RST):

```bash
tcpdump 'tcp[tcpflags] & tcp-rst != 0'
```

### Capture traffic with a specific TCP flag (FIN):

```bash
tcpdump 'tcp[tcpflags] & tcp-fin != 0'
```

DANIEL MIESSLER

```bash
tcpdump 'tcp[tcpflags] & tcp-urg != 0'
```

### Capture traffic with a specific TCP flag (PSH):

```bash
tcpdump 'tcp[tcpflags] & tcp-push != 0'
```

### Capture traffic with a specific TCP flag (ALL):

```bash
tcpdump 'tcp[tcpflags] = 0x01'
```

### Capture traffic with a specific TCP flag (NONE):

```bash
tcpdump 'tcp[tcpflags] = 0x00'
```

### Capture traffic with a specific TCP flag (SYN/ACK):

```bash
tcpdump 'tcp[tcpflags] = 0x12'
```

### Capture traffic with a specific TCP flag (SYN/RST):

```bash
tcpdump 'tcp[tcpflags] = 0x14'
```

### Capture traffic with a specific TCP flag (SYN/FIN):

```bash
tcpdump 'tcp[tcpflags] = 0x11'
```

### Capture traffic with a specific TCP flag (PSH/ACK):

DANIEL MIESSLER

### Capture traffic with a specific IP fragment offset:

BASH

```bash
tcpdump 'ip[6:2] & 0x1fff != 0'
```

### Capture traffic with a specific IP TTL:

BASH

```bash
tcpdump 'ip[8] = 128'
```

### Capture traffic with a specific IP DSCP:

BASH

```bash
tcpdump 'ip[1] & 0xfc >> 2 = 46'
```

### Capture traffic with a specific IP ECN:

BASH

```bash
tcpdump 'ip[1] & 0x03 = 3'
```

### Capture traffic with a specific TCP sequence number:

BASH

```bash
tcpdump 'tcp[4:4] = 12345678'
```

### Capture traffic with a specific TCP acknowledgement number:

BASH

```bash
tcpdump 'tcp[8:4] = 87654321'
```

### Capture traffic with a specific TCP source port range:

BASH

```bash
tcpdump 'tcp[0:2] > 1023 and tcp[0:2] < 65536'
```

DANIEL MIESSLER

BASH

```bash
tcpdump 'tcp[2:2] > 1023 and tcp[2:2] < 65536'
```

These examples should provide a solid foundation for using `tcpdump` to analyze network traffic.

Happy hunting!

*-Daniel*

SHARE Post LinkedIn HN Hacker News Reddit Facebook Forward

FOLLOW Get The Newsletter Follow On X Subscribe On YouTube Follow On LinkedIn

SEARCH *Search all 3029 posts across 29.3190239366 years and 20 tags...*

THIS POST WAS TAGGED WITH: cybersecurity technology tutorial top

HOME · BLOG · ARCHIVES · ABOUT

© 1999 — 2026 DANIEL MIESSLER. ALL RIGHTS RESERVED.

DANIEL MIESSLER